Advanced AI Medical Intelligence Platform

Complete Technical & System Architecture Documentation

Author: Anirudh Cherukuri

1. Project Overview & Objectives

The **Advanced AI Medical Intelligence Platform** is a full-stack, production-grade medical artificial intelligence application designed to assist radiologists, clinical technicians, and medical practitioners by providing automated diagnostic decision support for Chest Radiographs (X-Rays).

Core Platform Objectives

- **Thoracic Pathology Classification:** Classify input radiographs into 3 distinct diagnostic categories: **Normal**, **Pneumonia**, and **COVID-19**.
- **Anatomical Validation:** Reject invalid file formats, corrupted files, non-medical photos, and non-thoracic radiographs (Abdominal X-Rays, Hand X-Rays, CT Scans) before running deep learning inference.
- **Explainable AI (XAI):** Generate pixel-level feature attribution heatmaps using Grad-CAM, overlaid on original radiographs with interactive opacity controls and spatial metrics (affected coverage %, peak intensity location, severity score).
- **LLM Radiology Report Synthesis:** Automatically synthesize structured radiology evaluation reports using Google Gemini 1.5 Flash API, backed by an offline Clinical Logic Engine for zero-downtime fallback.
- **Audit Logging & PDF Reporting:** Store all diagnostic evaluations in an SQLite database and compile printable, professional PDF radiology reports using ReportLab.

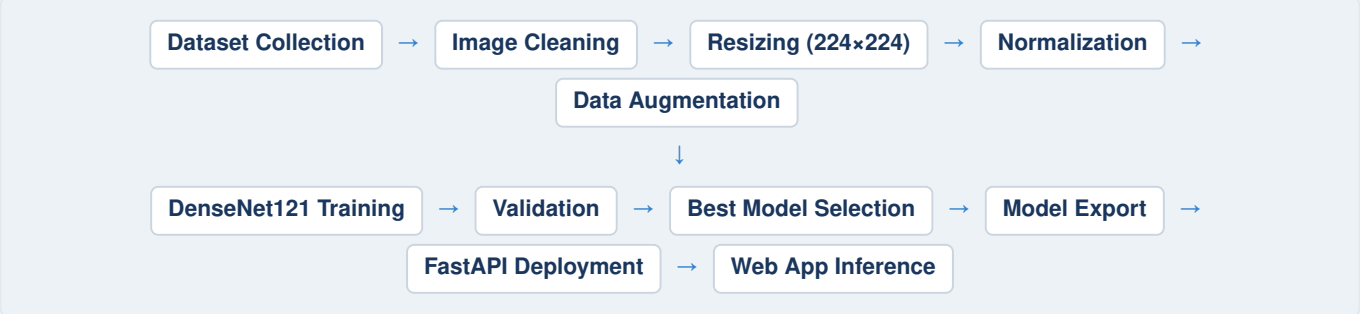
2. Dataset Description

The model was trained using the **Kaggle COVID-19 Radiography Dataset**. The dataset contains three diagnostic categories: **COVID-19**, **Normal**, and **Viral Pneumonia**.

- **Preprocessing & Normalization:** Images were cleaned, resized to 224×224 pixels, and normalized using ImageNet statistics.
- **Data Augmentation:** Applied random flips, rotations, and affine transformations to improve model generalization.
- **Dataset Splitting:** Divided into training, validation, and testing subsets.
- **Deployment Artifact:** The best performing checkpoint was exported as `best_model.pth` and integrated into the FastAPI backend.

3. CNN Development Pipeline

The end-to-end development and deployment workflow spans data acquisition to production inference:



4. DenseNet121 Architecture

DenseNet121 was selected because of its dense connectivity pattern that allows each layer to receive feature maps from all preceding layers, encouraging feature reuse and improved gradient propagation across deep feature blocks.

- **Network Topology:** Initial convolution layer, four Dense Blocks separated by Transition Layers, Global Average Pooling, and a custom fully connected classification head.
- **Target XAI Layer:** `densenet.features.denseblock4.denselayer16`
- **Custom Classification Head Structure:**

```
Sequential(
    BatchNorm1d(1024), Dropout(0.5),
    Linear(1024, 512), ReLU(inplace=True),
    BatchNorm1d(512), Dropout(0.5),
    Linear(512, 256), ReLU(inplace=True),
    Dropout(0.5),
    Linear(256, 3) # Target Classes: ['COVID-19', 'Normal', 'Pneumonia']
)
```

A **Softmax** layer converts raw output logits into class probabilities used for diagnostic prediction.

5. Kaggle Training Workflow

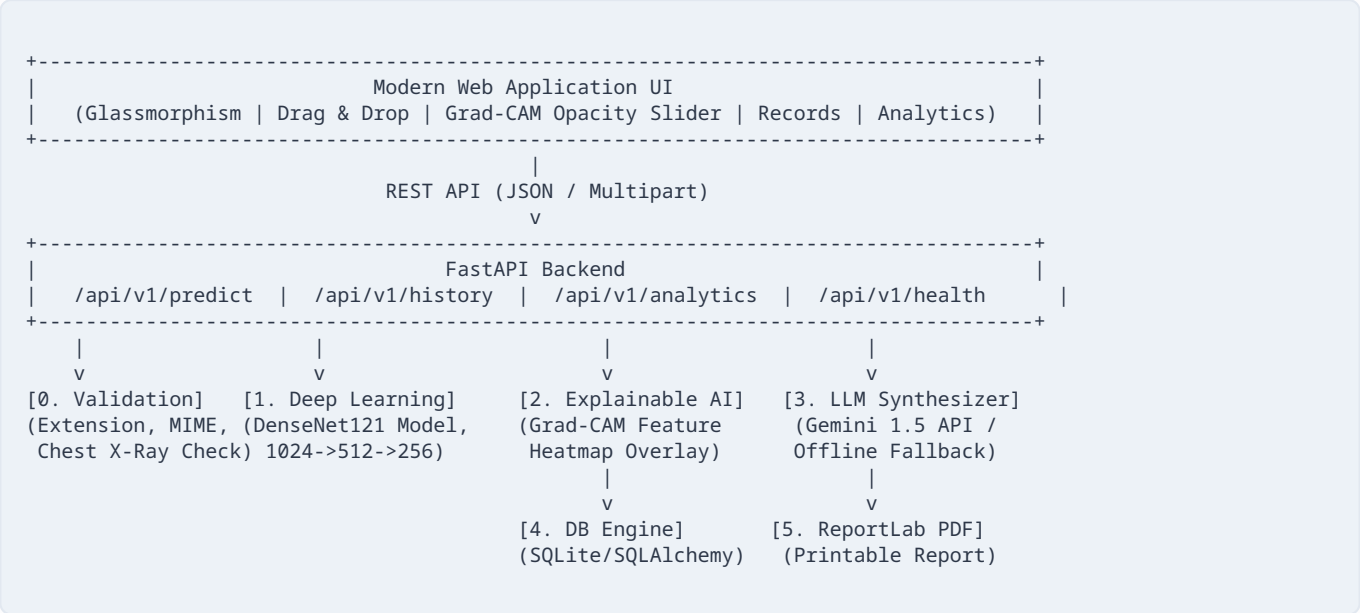
Training was performed within the **Kaggle GPU Environment**. After preprocessing and augmentation, DenseNet121 was fine-tuned for chest X-ray classification. Validation accuracy was monitored during training across a target of up to 100 epochs. The model achieved its peak validation accuracy at Epoch 36, at which point the best-performing model checkpoint (`best_model.pth`) was automatically saved. Performance metrics including confusion matrix, ROC curves, Grad-CAM outputs, and training history were exported and integrated into the analytics dashboard.

6. Hyperparameter Configuration

Hyperparameter / Parameter	Configuration Value
Framework	PyTorch
Architecture	DenseNet121
Optimizer	Adam
Learning Rate	0.0001 (1e-4)
Loss Function	CrossEntropyLoss
Batch Size	32
Input Resolution	224 × 224 pixels
Diagnostic Classes	COVID-19, Normal, Pneumonia
Output Artifact	`best_model.pth`

7. Full System Architecture

The platform follows a decoupled, asynchronous multi-layer design to cleanly separate validation, inference, explainability, LLM orchestration, storage, and presentation layers:



8. Pre-Inference Validation Layer & Anatomical Verification

Located in `backend/utils/validation.py`, the validation layer guarantees that the system only processes valid Thoracic Chest Radiographs through a 4-step sequence:

1. **File Extension & MIME Type Check (`validate_file`):** Allowed extensions are `.png`, `.jpg`, `.jpeg` with MIME types `image/png` and `image/jpeg`. Maximum allowed size is 15MB.
2. **Image Integrity Verification (`validate_image`):** Reads image bytes and executes `PIL.Image.verify()`. Rejects empty (0-byte) or corrupted files with HTTP 400.
3. **Anatomical Chest X-Ray Verification (`validate_chest_xray`):**
   - *Color Variance Check:* Computes RGB channel variance $mean(|R-G| + |G-B|)$. If > 5.0, rejects standard color photographs.
   - *Structural Background Void Check:* Evaluates black background ratio (<25 intensity). If >70%, rejects extremity/ hand/foot X-Rays.
   - *Circular CT FOV Check:* Analyzes corner intensity versus central region to identify circular field-of-view axial Head/ Brain CT Scans.
   - *Bilateral Thoracic Pulmonary Lung Cavity Verification:* Analyzes upper-mid thoracic lung zones versus central mediastinum/spine contrast. Rejects abdominal X-rays lacking upper dark lung cavities.

9. Explainable AI (Grad-CAM) Engine

Located in `backend/xai/gradcam.py`, Grad-CAM calculates the gradient of score y^c for class c with respect to feature maps A^k of the final dense convolutional layer:

$$\alpha_k^c = (1 / Z) \sum_i \sum_j (\partial y^c / \partial A_{i,j}^k)$$

The coarse class activation map is synthesized using a rectified linear combination:

$$L_{Grad\text{-}CAM}^c = ReLU(\sum_k \alpha_k^c A^k)$$

Visualization & Metrics: Heatmaps are color-mapped via OpenCV `COLORMAP_JET` and blended at 50% opacity over the original radiograph. Spatial metrics extracted include affected coverage %, peak intensity location, and regional severity score.

10. LLM Radiology Report Synthesis & Fallback Engine

Located in `backend/llm/report_generator.py`:

- **Primary Engine:** Google Gemini 1.5 Flash API (`google-genai` SDK).
- **Prompt Structure:** Combines patient demographics, model confidence, class probabilities, and Grad-CAM spatial metrics to generate structured Clinical Impressions, Radiological Findings, XAI Interpretations, and Follow-up Plans.
- **Offline Fallback Engine:** An offline Clinical Logic Engine generates structured reports if network access or API key issues occur, ensuring 100% service continuity.

11. Database Schema & Audit Trail

Located in `backend/database/models.py` (SQLite via SQLAlchemy):

```
CREATE TABLE scan_records (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    scan_id VARCHAR UNIQUE INDEX,
    patient_id VARCHAR INDEX,
    patient_name VARCHAR,
    age INTEGER,
    gender VARCHAR,
    scan_type VARCHAR,
    original_image_path VARCHAR,
    heatmap_image_path VARCHAR,
    overlay_image_path VARCHAR,
    predicted_class VARCHAR INDEX,
    confidence FLOAT,
    probabilities_json TEXT,
    spatial_metrics_json TEXT,
    activation_coverage_pct FLOAT,
    severity_score FLOAT,
    report_text TEXT,
    risk_level VARCHAR,
    report_source VARCHAR,
    created_at DATETIME INDEX
);
```

12. ReportLab PDF Report Generation

Located in `backend/utils/pdf_generator.py`:

Generates individual patient Clinical Radiology Reports featuring header metadata, demographic tables, primary diagnostic badges, side-by-side radiograph vs. Grad-CAM visual evidence, and formal medical disclaimers.

13. Web Application & UI/UX Studio

Located in `frontend/`:

- **Diagnostic Studio Tab:** Drag-and-drop uploader, sample preset buttons, interactive opacity slider, and live probability progress bars.
- **Glassmorphism Warning Modal:** Displays interactive warnings whenever an invalid or non-Chest X-Ray image is rejected.
- **Patient Records & Analytics Tabs:** Searchable audit history table, system performance indicators, and training/evaluation charts.

14. API Specifications & OpenAPI Endpoints

Endpoint	Method	Description
`/api/v1/predict`	`POST`	Full Pipeline: Validation → Inference → Grad-CAM → LLM → DB → PDF
`/api/v1/history`	`GET`	Retrieve searchable/filterable patient audit history records
`/api/v1/history/{scan_id}`	`GET`	Fetch specific scan record details
`/api/v1/analytics`	`GET`	Fetch system performance metrics and historical stats
`/api/v1/health`	`GET`	API health check and GPU/CPU resource status

15. Installation, Setup, & Execution Guide

Because model training was conducted offline on Kaggle (saving optimal weights at Epoch 36 as `best_model.pth`), local execution only requires:

1. **Clone Repository:**

```
git clone https://github.com/anirudhcherukuri/AI_Medical_platform.git
cd AI_Medical_platform
```

2. **Install Dependencies:**

```
pip install -r requirements.txt
```

3. **Verify Pre-trained Weights:** Ensure `best_model.pth` is placed in `backend/weights/`.
4. **Initialize Database & System Assets:** Initialize SQLite DB schemas.
5. **Launch Application Server:**

```
uvicorn backend.main:app --reload
```